

Database Management System

Keyword

- Data: raw facts and figure
- Information: Meaningful data
- Database: collection of interrelated data
- DBMS: complex set of software program that control the organization, storage, management and retrieval of data in a Database.

Data → **Information** → **Database** → **System**

- Example: Ms- Access, MySql, SQL Server, Oracle

Introduction

A Database Management System (DBMS) is a software system that is designed to manage and organize data in a structured manner. It allows users to create, modify, and query a database, as well as manage the security and access controls for that database.

Key Features of DBMS

- **Data modeling:** Provides tools for creating and modifying data models
- **Data storage and retrieval:** Responsible for storing and retrieving data, various methods for searching and querying the data.
- **Concurrency control:** Provides mechanisms for controlling concurrent access to the database, to ensure that multiple users can access the data without conflicting with each other.
- **Data integrity and security:** Provides tools for enforcing data integrity and security constraints
- **Backup and recovery:** Provides mechanisms for backing up and recovering the data in the event of a system failure.
- **DBMS can be classified into two types:** Relational Database Management System (RDBMS) and Non-Relational Database Management System (NoSQL or Non-SQL)
- **RDBMS:** Data is organized in the form of tables and each table has a set of rows and columns.

Uses of Database

- Banking: All Transactions
- Airlines: Reservations, Schedules
- Universities: Registration, Examination
- Sales: Customers, Products, Purchases
- Manufacturing: Production, Inventory, Orders
- Human Resources: Employee Records, Salaries, Tax deduction

Comparison between File System and DBMS

File System

- The file system is a way of arranging the files in a storage medium within a computer
- Redundant data can be present in a file system
- It doesn't provide Inbuilt mechanism for backup and recovery of data if it is lost.
- There is no efficient query processing in the file system.
- Less data consistency in the file system.
- File systems provide less security in comparison to DBMS.
- Only one user can access data at a time.
- It give details of storage and representation of data

DBMS

- DBMS is software for managing the database
- No redundant database
- It provides in house tools for backup and recovery of data even if it is lost
- Efficient query processing is there in DBMS.
- More data consistency because of the process of normalization.
- More security mechanisms as compared to file systems.
- Multiple users can access data at a time.
- It hides the internal details of Database

Drawback of File System

- Data Redundancy
- Inconsistency of data
- Difficult Data Access
- Unauthorized Access
- No Concurrent Access
- No Backup and Recovery

Advantages of Database

- Data Organization
- Data Integrity
- Concurrent access
- Data Security
- Backup and Recovery
- Data sharing

Disadvantages of Database

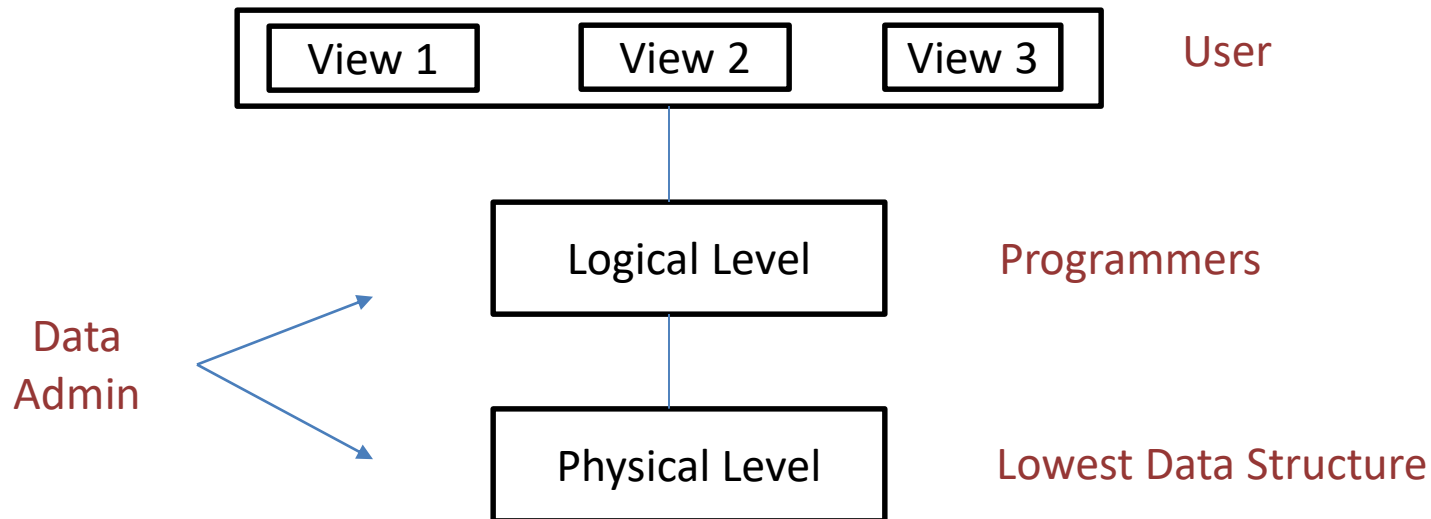
- Complexity
- Performance overhead
- Scalability
- Cost
- Limited Use Cases

View of Data

- Data Abstraction
- Instances and Schema
- Data Independency
- Data Model

Data Abstraction

- **Data Abstraction** : shows only necessary information to the user and hides background details.



- Physical Level (Internal Level)
 - ✓ contains all the complex data structures and the data accessing methods defined.
 - ✓ lowest level of data abstraction in a DBMS.
 - ✓ database administrator who decides **how** the data is to be stored in these physical hard drives.
- Logical Level (Conceptual Level)
 - ✓ After taking the raw data from the physical or internal level, the structure of the data is defined at the logical or conceptual level.
 - ✓ Blueprint of the raw data
 - ✓ This layer does not have any information about how the end user will view the data.
- View Level (External Level)
 - ✓ This is what an end-user gets to see.
 - ✓ He/she does not get the entire database, but depending on the queries made from the front-end the user gets to see the data.
 - ✓ It may be a single data from the entire database or a collection of data in tabular format.
 - ✓ Multiple views of the same data are available to the user, the representation can be a table, a graph, or a pie chart.
 - ✓ View Level is the highest level of data abstraction in DBMS.

Instances and Schema

- **Instances:** Collection of information stored in a database at a particular moment
- **Schema:** Overall design of the database
 - ✓ Physical Schema: The internal schema describes how the data will be physically stored and accessed, using the facilities provided by a particular DBMS.
 - ✓ Logical Schema: The Conceptual Schema describes the stored data in terms of the data model of the DBMS
 - ✓ View Schema: The External Schema specify views that enable different users of the data to see it in different ways.

Data Independence

- **Physical Data Independence:** Ability to modify the physical schema without causing application program to be rewritten.

Modification at the physical level occasionally necessary to improve performance

- **Logical Data Independence :** Ability to modify logical schema without causing changes in physical system.

Logical data independence is more difficult to achieve physical data independence since application program heavily dependent on logical structure of data they access

Data Model

- Collection of conceptual tools for describing data and data relationship, data semantics and consistency constraints.
- A data model is provides the way to describe the design of a database at physical, logical and view level.
 - ✓ Relational Model
 - ✓ Entity Relationship Model
 - ✓ Object Based Model
 - ✓ Semi-Structured Data Model
 - ✓ Network Model
 - ✓ Hierarchy Model}

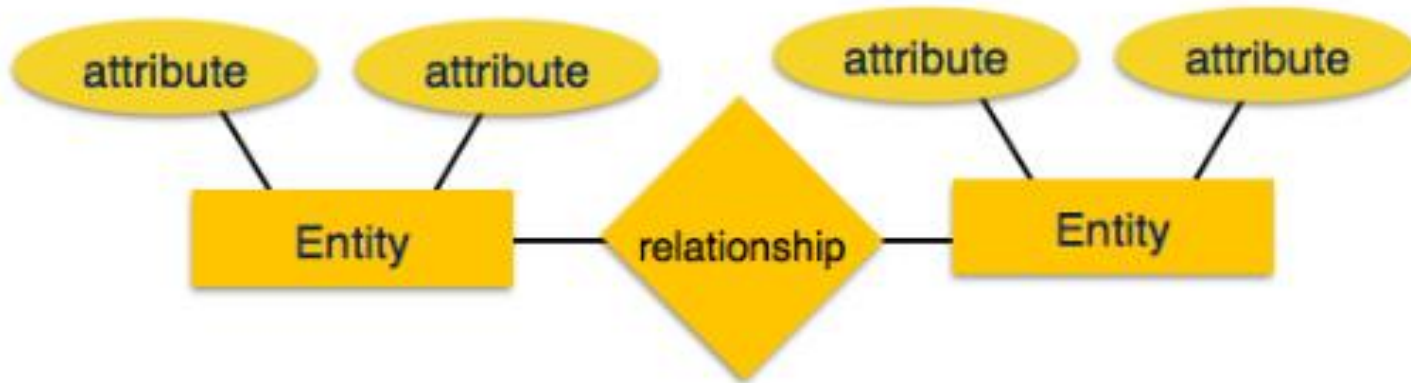
1. Relational Model

- The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name.
- The relational model represents how data is stored in Relational Databases.
- Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE, and AGE shown in the table.

Table Student				
ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI		18

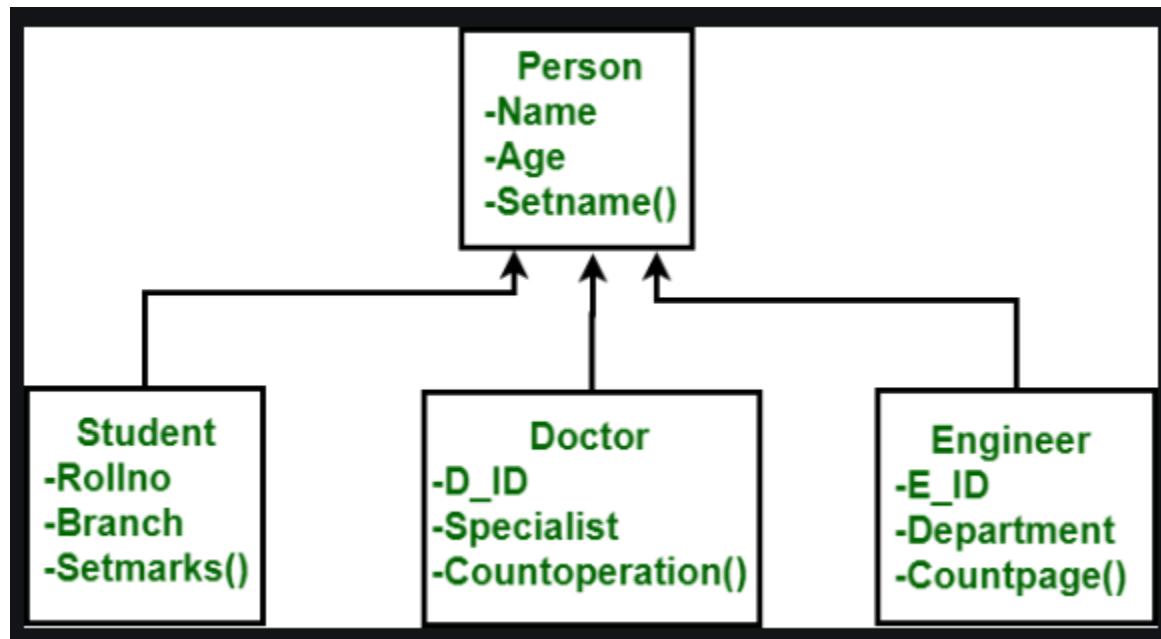
2. Entity-Relationship Model

- Entity-Relationship (ER) Model is based on the notion of real-world entities and relationships among them.
- While formulating real-world scenario into the database model, the ER Model creates **entity set**, **relationship set**, **general attributes** and **constraints**.
- ER Model is best used for the conceptual design of a database.



3. Object based Model

- In Object Oriented Data Model, data and their relationships are contained in a single structure which is referred as object in this data model.
- In this, real world problems are represented as objects with different attributes.
- All objects have multiple relationships between them.
- Basically, it is combination of Object Oriented programming and Relational Database Model

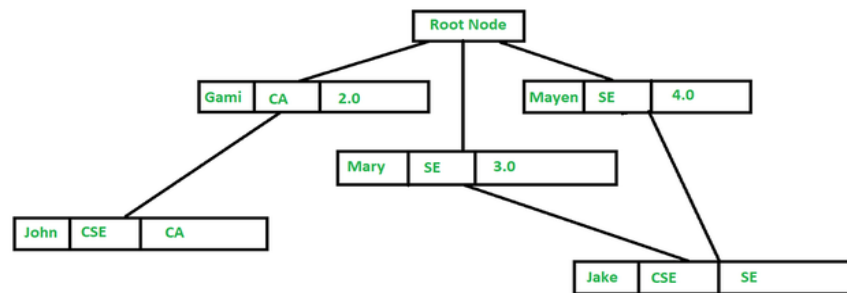


4. Semi-Structured Model

- Semi-structured data is a type of data that is not purely structured, but also not completely unstructured.
- It contains some level of organization or structure, but does not conform to a rigid schema or data model, and may contain elements that are not easily categorized or classified.
- For example, an XML document might contain tags that indicate the structure of the document, but may also contain additional tags that provide metadata about the content, such as author, date, or keywords.
- Other examples of semi-structured data include JSON, which is commonly used for exchanging data between web applications, and log files, which often contain a mix of structured and unstructured data.

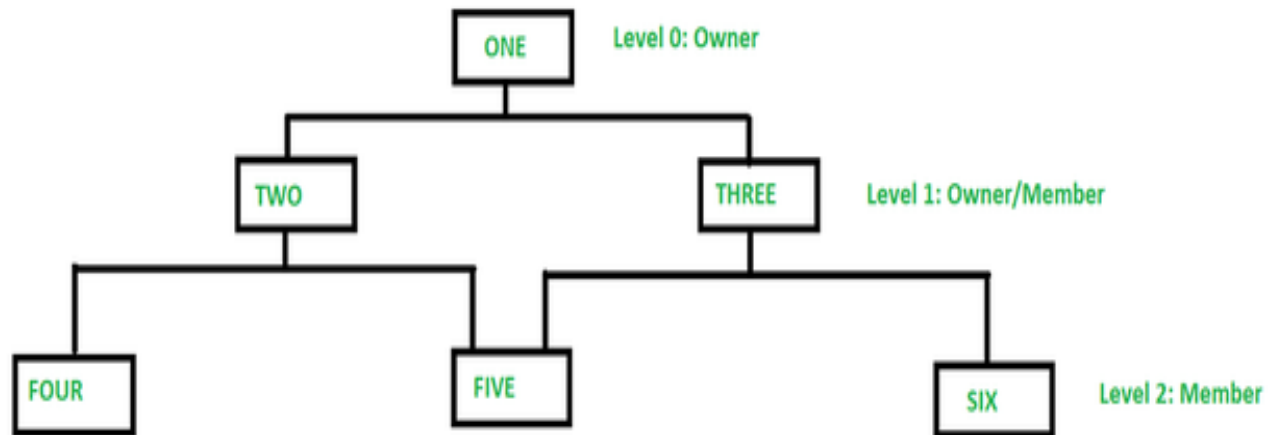
5. Hierarchical Model

- In a hierarchical model, data are viewed as a collection of tables, or we can say segments that form a hierarchical relation.
- In this, the data is organized into a tree-like structure where each record consists of one parent record and many children.
- In the hierarchical model, segments pointed to by the logical association are called the child segment and the other segment is called the parent segment.
- If there is a segment without a parent is then that will be called the **root** and the segment which has no children are called the **leaves**.
- The main disadvantage of the hierarchical model is that it can have one-to-one and one-to-many relationships between the nodes.



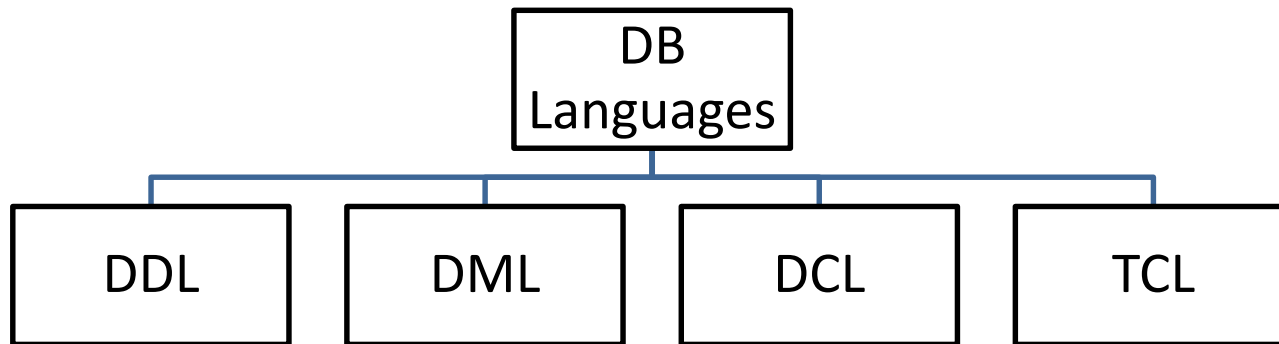
6. Network Model

- The Network Model in a Database Management System (DBMS) is a data model that allows the representation of many-to-many relationships in a more flexible and complex structure compared to the Hierarchical Model.
- It uses a graph structure consisting of nodes (entities) and edges (relationships) to organize data, enabling more efficient and direct access paths.



Database Languages

- A DBMS has appropriate languages and interfaces to express database queries and updates.
- Database languages can be used to read, store and update the data in the database.



Data Definition Languages

- DDL stands for **Data Definition Language**.
- It is used to define database structure or pattern.
- It is used to create schema, tables, indexes, constraints, etc. in the database.
- Using the DDL statements, you can create the skeleton of the database.
- Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.
 - ✓ **Create:** It is used to create objects in the database.
 - ✓ **Alter:** It is used to alter the structure of the database.
 - ✓ **Drop:** It is used to delete objects from the database.
 - ✓ **Truncate:** It is used to remove all records from a table.
 - ✓ **Rename:** It is used to rename an object.
 - ✓ **Comment:** It is used to comment on the data dictionary.

Data Manipulation Language

- DML stands for **Data Manipulation Language**.
- It is used for accessing and manipulating data in a database.
- It handles user requests.
 - ✓ **Select:** It is used to retrieve data from a database.
 - ✓ **Insert:** It is used to insert data into a table.
 - ✓ **Update:** It is used to update existing data within a table.
 - ✓ **Delete:** It is used to delete all records from a table.
 - ✓ **Merge:** It performs UPSERT operation, i.e., insert or update operations.
 - ✓ **Call:** It is used to call a structured query language or a Java subprogram.
 - ✓ **Explain Plan:** It has the parameter of explaining data.
 - ✓ **Lock Table:** It controls concurrency.

Types of DML

Procedural DML: It requires a user to specify what data are needed and how to get those data.

Non-Procedural DML: It requires a user to specify one data are needed without specifying how to get those data.

Non-Procedural DML are easy to learn and easy to use.

Data Control Language

- DCL stands for **Data Control Language**.
- It is used to retrieve the stored or saved data.
- The DCL execution is transactional.
- It also has rollback parameters.
 - ✓ **Grant:** It is used to give user access privileges to a database.
 - ✓ **Revoke:** It is used to take back permissions from the user.

Transaction Control Language

- TCL is used to run the changes made by the DML statement.
- TCL can be grouped into a logical transaction.
 - ✓ **Commit:** It is used to save the transaction on the database.
 - ✓ **Rollback:** It is used to restore the database to original since the last Commit.

Types of Constraints

- **Domain Constraints:** A domain of possible values must be associated with every attribute.

For Eg. int, char, time

- **Referential Integrity:** A value appear in one relation for a given set of attribute also appears in a certain set of attributes in another relation.
- **Authorization:** Differentiate in a type of access that permits the user to access various data values in database.

For Eg. Read, Insert, Update, Delete

- **Assertion:** It is any condition that the DB must always satisfy when an assertion is created the system test in for validity. If an assertion is valid then any future modification to the DB allow.

Database Design

- **Database Users:** There are four types of Database Users
 - a) Naive Users
 - b) Application Programmers
 - c) Sophisticated Users
 - d) Specialized Users

Database Users

a) Naive Users

- They are **unsophisticated users** who interact with the system by interacting one of the application programs that have been written previously.
- User interface for naïve user is a form interface where the user can fill fields of the form.
- They may simply read reports generated by the database.

b) Application Programmers

- They are **computer professional** who write programs for specific application.
- Application Programmers can choose from many tools to develop user interface.
- Rapid Application Development(RAD) are tools that enable an application programmer to construct forms and report with minimal programming efforts.

Database Users

c) Sophisticated Users

- They interact with the system without writing programs instead they form their request either using a **DB query language** or using tools such as data analysis system.
- Analyst who submit queries to explore data in database fall in this category.

d) Specialized Users

- They are sophisticated users who write **specialized DB application** that do not fit into the traditional data processing frame work.
- Uses Computer Added Design System(CAD) that store data with complex data types. For eg. Graphics and Audio Data

Database Administrator

- A person who has **central control** over the system is known as Database Administrator (DBA).
- Functions of DBA are:
 - a) Schema Function:** The DBA create the original database schema by executing a set of data definition statement in DDL.
 - b) Storage structure and access method definition:** DBA defines the structure to store data in DBMS and ways through which data from the database can be accessed.
 - c) Schema and physical organisation modification:** DBA carries out the schema and physical organisation to reflect the changing needs of the organization or to alter the physical organisation to improve performance.
 - d) Granting of Authorization for data access:** DBA grants different type of access authorization to user for accessing different parts of DBMS. The authorization information is kept in a special system structure that the DBMS concert whenever someone attempts to access data in system.
 - e) Routine Maintenance:** Periodically backing up the database, ensuring that enough free disk space is available for normal operation and upgrading disk space is require and monitoring job running on the DB and ensuring that performance is not degraded by very expensive task submitted by users.

Database Overall Structure

Components of DBMS are broadly classified as follows :

1. Query Processor :

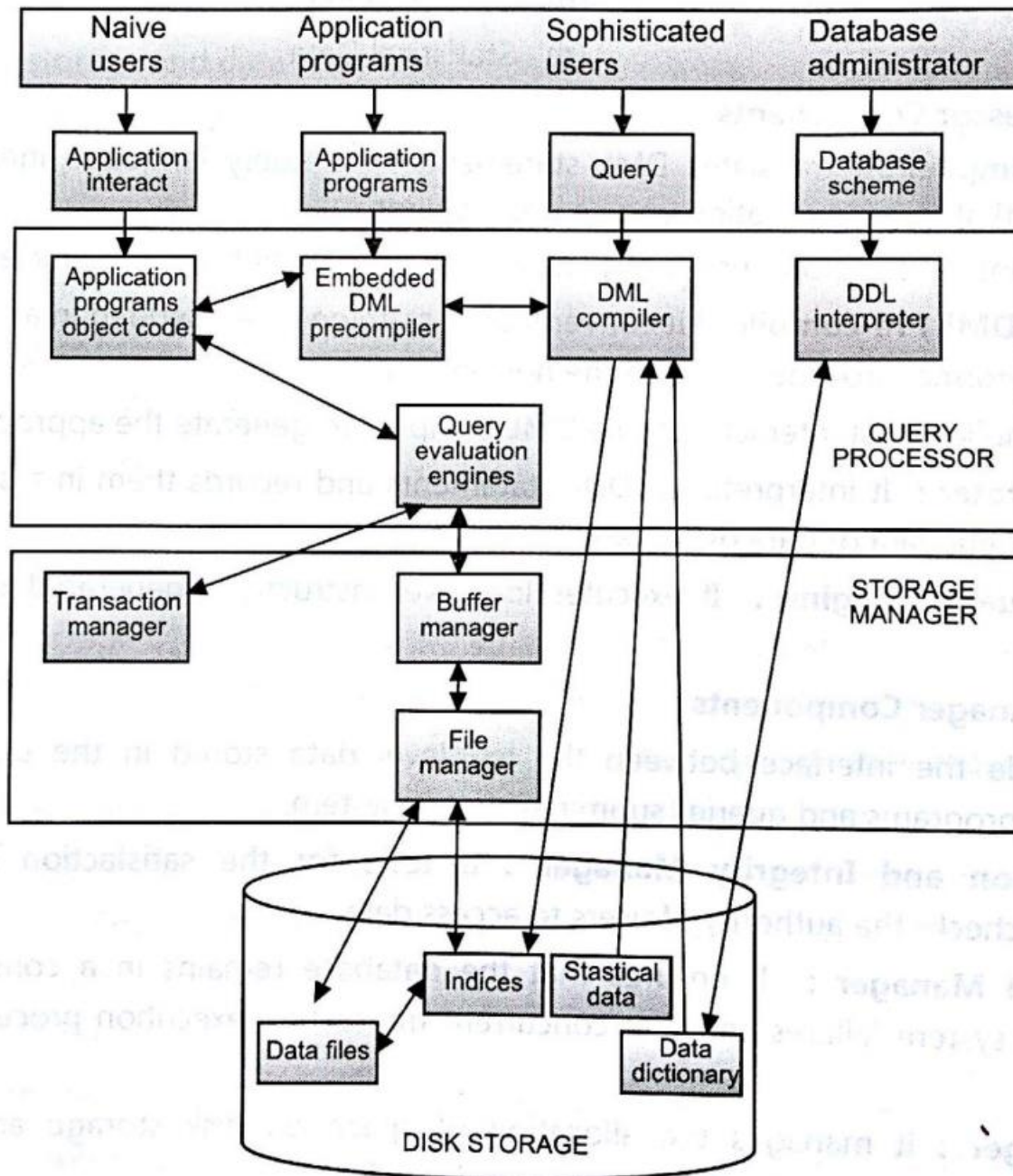
- (a) DML Compiler
- (b) Embedded DML pre-compiler
- (c) DDL Interpreter
- (d) Query Evaluation Engine

2. Storage Manager :

- (a) Authorization and Integrity Manager
- (b) Transaction Manager
- (c) File Manager
- (d) Buffer Manager

3. Disk storage:

- (a) Data Files
- (b) Data Dictionary
- (c) Indices
- (d) Statistical Data



Disk Storage

Following data structures are required as a part of the physical system implementation.

a) **Data Files** : It stores the database.

b) **Data Dictionary** : It stores meta data (data about data) about the structure of the database.

c) **Indices** : Provide fast access to data items that hold particular values.

d) **Statistical Data** : It stores statistical information about the data in the database. This information is used by query processor to select efficient ways to execute query.

Storage Manager Components

They provide the interface between the low-level data stored in the database and application programs and queries submitted to the system.

- a) **Authorization and Integrity Manager** : It tests for the satisfaction of integrity constraints checks the authority of users to access data.
- b) **Transaction Manager** : It ensures that the database remains in a consistent state despite the system failures and that concurrent transaction execution proceeds without conflicting.
- c) **File Manager** : It manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- d) **Buffer Manager** : It is responsible for fetching data from disk storage into main memory and deciding what data to reside in cache memory.

Query Processor Components

- a) **DML Pre-compiler** : It translates DML statements in a query language into low level instructions that query evaluation engine understands. It also attempts to transform user's request into an equivalent but more efficient form.
- b) **Embedded DML Pre-compiler** : It converts DML statements embedded in an application program to normal procedure calls in the host language. The Pre-compiler must interact with the DML compiler to generate the appropriate code.
- c) **DDL Interpreter** : It interprets the DDL statements and records them in a set of tables containing meta data or data dictionary.
- d) **Query Evaluation Engine** : It executes low-level instructions generated by the DML compiler.

Entity Relationship Model

ER Model

- ER Model is a conceptual data model that views the real world as entities and relationships.
- It is a graphical representation of entities and their relationship to each other.
- **Utility of ER Model**
 - It maps well to the relational model. ER Model can easily be transformed into relational tables.
 - It is simple and easy to understand with a minimum of training. Therefore the model can be used by the database designer to communicate the design to the end user.
 - It can be used as a design plan by the database developer to implement a data model in specific database management software.

ER Model

- **Entity:** An entity is a thing or an object in the real world i.e. distinguishable from all other objects.

For Eg. Each student in the university is an entity.

- **Entity Set:** An entity set is a set of entities of the same type that share the same property of attribute.

For Eg. An entity set of all student represent set of all students

- **Attribute:** An entity is represented by a set of Attributes. Attributes are descriptive property posses by each member of an entity set.

For Eg. Each student in entity set student may have ID, Name, Course, etc.

- **Relationship:** A relationship is associated among several entities.
- **Relationship Set:** A relationship set is a set of relation of same type.

For Eg. If $E_1, E_2, \dots, E_n$ are entity sets then a relationship set R is a subset of $(e_1, e_2, \dots, e_n \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n)$ where $e_1, e_2, \dots, e_n$ are relationship.

- A relationship may also have attributes called **descriptive attribute**.

For Eg. Date at which student joins the course

Attribute

- An Attribute of an entity set is a function that maps from the entity set into the domain. For each attribute there is a set of permitted value called the domain or value set of that attribute.
- Several types of attributes we will use in an ER Model:

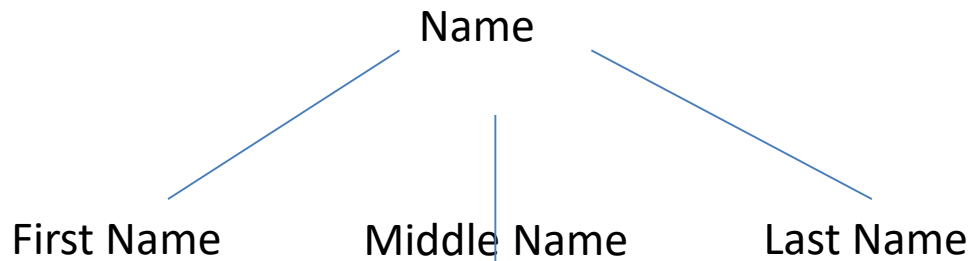
a) Simple and Composite Attribute:

Simple Attribute can not be divided into subparts.

For Eg. Student ID

Composite Attribute can be divided into sub parts.

For Eg. Student Name



b) Single Valued and Multi Valued Attribute:

Only one value is associated with an attribute.

For Eg. ID is unique for each student

Attribute with multiple value is known as Multi Valued Attribute.

For Eg. A person may have zero, one or several phone numbers.

c) Derived Attribute:

The value of this type of attribute can be derived from the value of other related attributes or entities.

For Eg. Age can be calculated using Date of Birth attribute.



d) Complex Attribute:

If any attribute has the combining property of multi values and composite attributes, then it is called a complex attribute. It means if one attribute is made up of more than one attribute and each attribute can have more than one value, then it is called a complex attribute.

For Eg., if a person has more than one office and each office has an address made from a street number and city. So the address is a composite attribute, and offices are multi valued attributes, So combining them is called complex attributes.

e) Key Attribute:

Those attributes which can be identified uniquely in the relational table are called key attributes.

For Eg., a student id is a unique attribute.

ER Symbols

•**Rectangles**: Rectangles represent Entities in the ER Model.

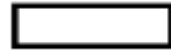
•**Ellipses**: Ellipses represent Attributes in the ER Model.

•**Diamond**: Diamonds represent Relationships among Entities.

•**Lines**: Lines represent attributes to entities and entity sets with other relationship types.

•**Double Ellipse**: Double Ellipses represent Multi-Valued Attributes.

•**Double Rectangle**: Double Rectangle represents a Weak Entity



Represents Entity



Represents Attribute



Represents Relationship



Links Attribute(s) to entity set(s) or Entity set(s) to Relationship set(s)



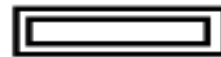
Represents Multivalued Attributes



Represents Derived Attributes



Represents Total Participation of Entity



Represents Weak Entity



Represents Weak Relationships



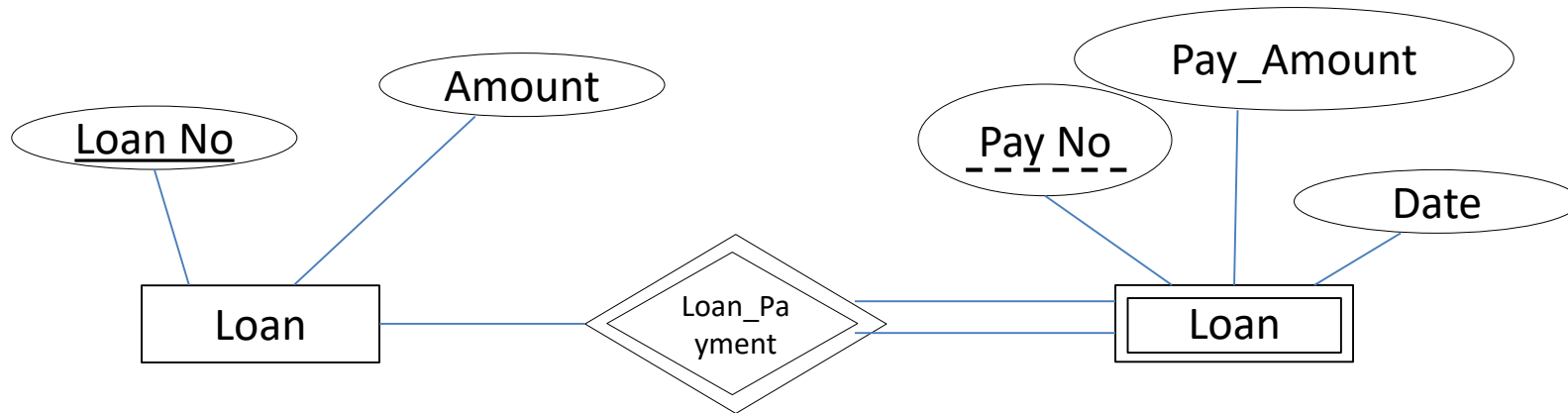
Represents Composite Attributes

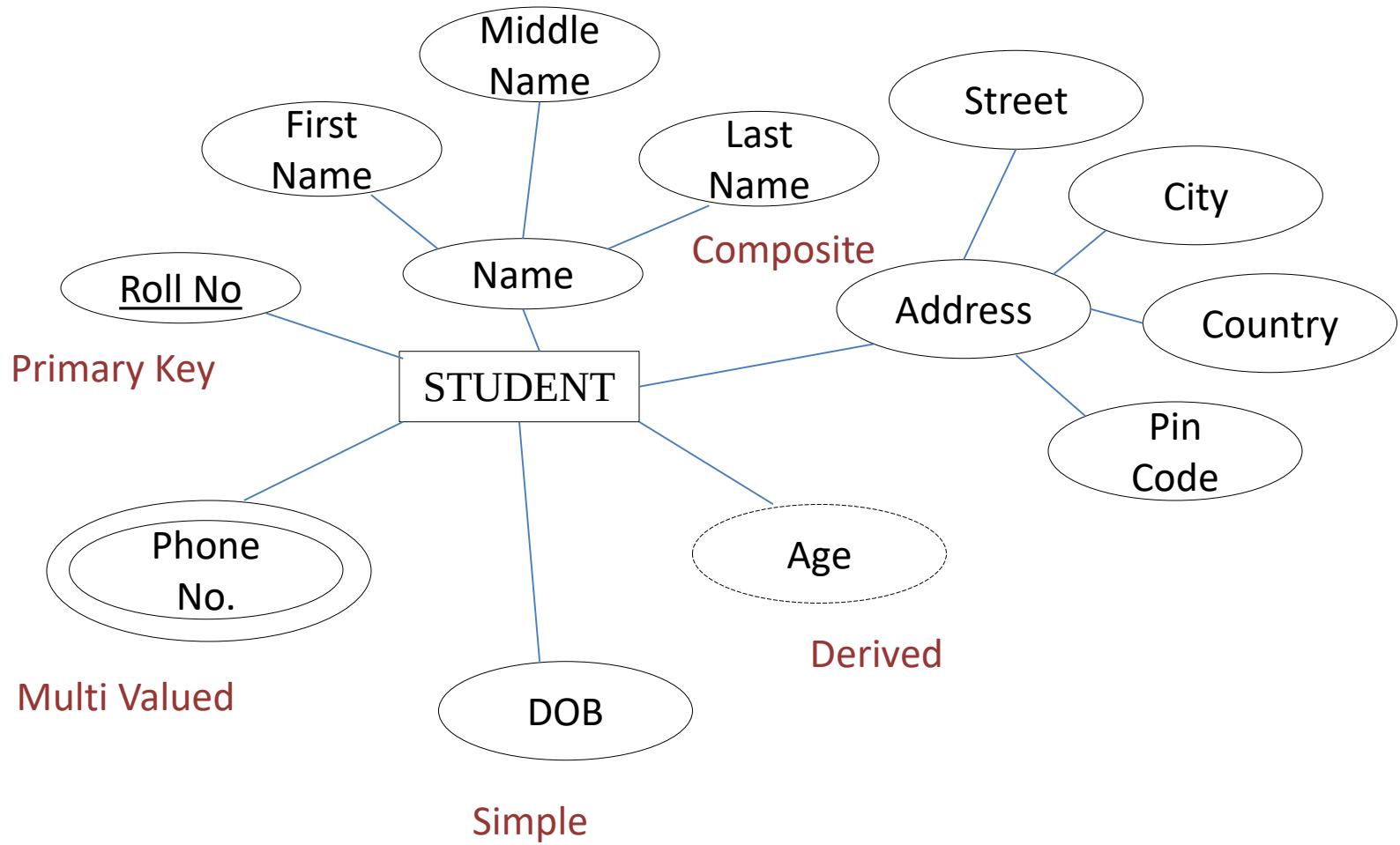


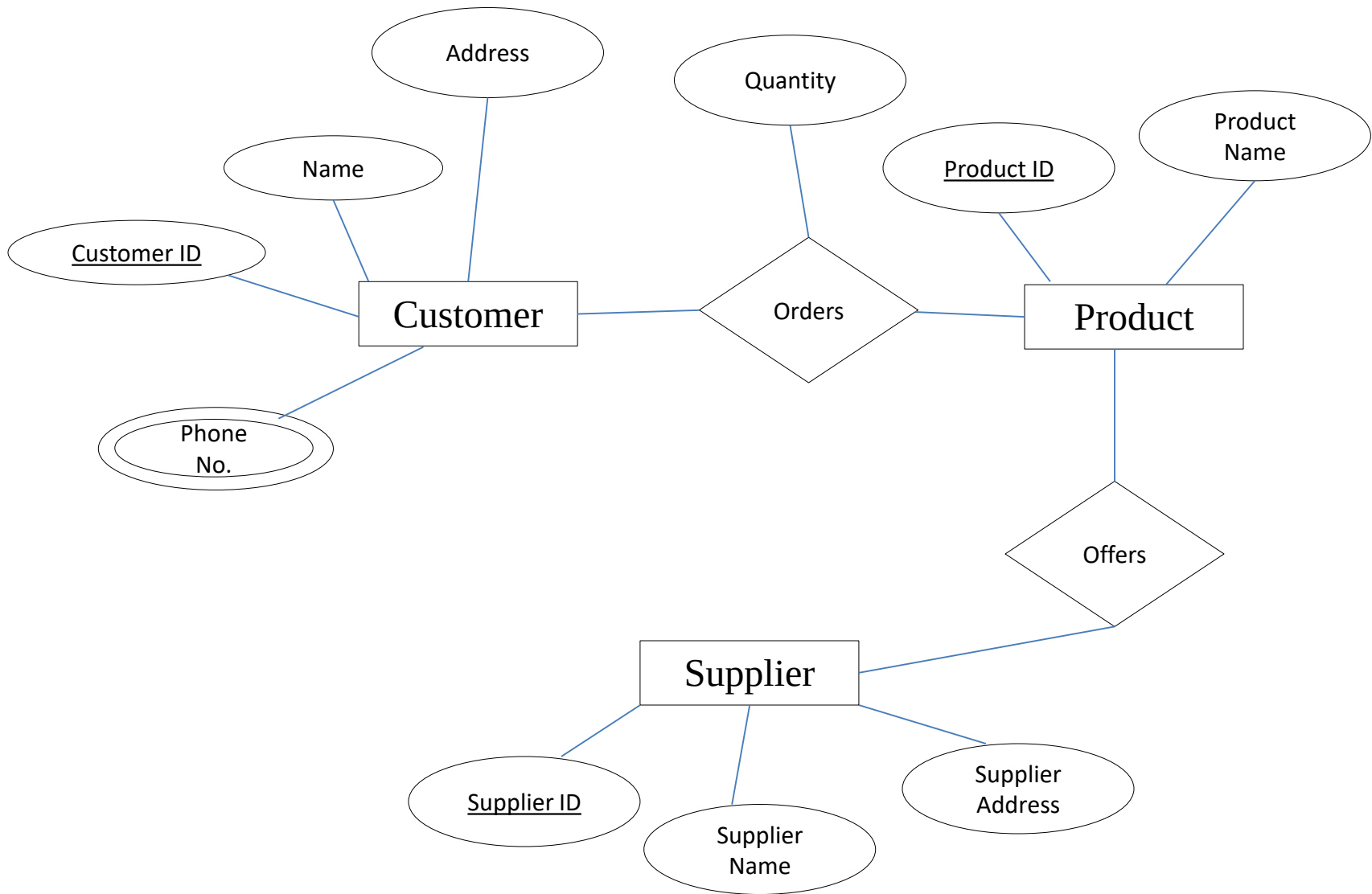
Represents Key Attributes / Single Valued Attributes

1. **Strong Entity:** A Strong Entity is a type of entity that has a key Attribute. Strong Entity does not depend on other Entity in the Schema. It has a primary key, that helps in identifying it uniquely, and it is represented by a rectangle. These are called Strong Entity Types.

2. **Weak Entity:** An Entity type has a key attribute that uniquely identifies each entity in the entity set. But some entity type exists for which key attributes can't be defined. These are called Weak Entity types.



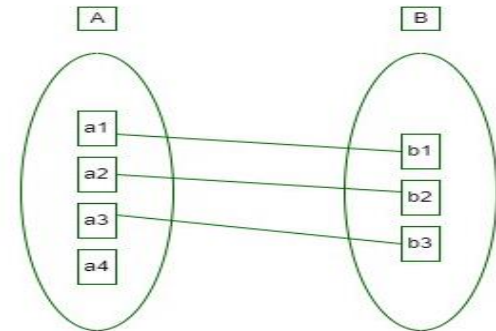




Mapping Constraints

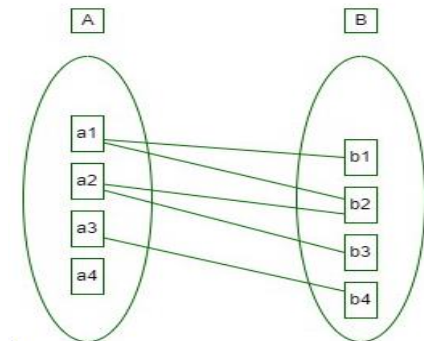
- It express the number of entities to which other entities can be associated with a relationship set.
- One – to – One: an entity in A is connected to at most one entity in B. Or we can say that a unit or item in B is connected to at most one unit or item in A.

Eg. Country-President



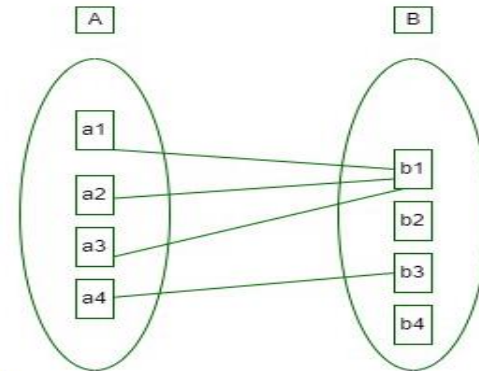
- One – to – Many: an entity in A is associated with any number of entities in B. Or we can say that one unit or item in B can be connected to at most one unit or item in A.

Eg. Teacher - Student



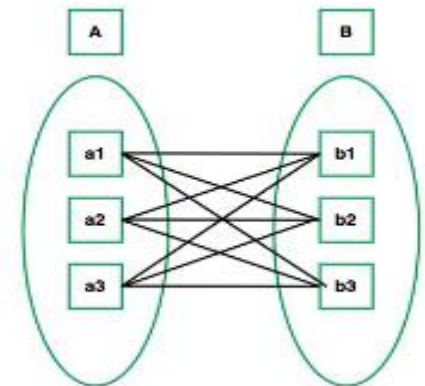
- Many – to – One: an entity in A is connected to at most one entity in B. Or we can say a unit or item in B can be associated with any number (zero or more) of entities or items in A.

Eg. State- Country



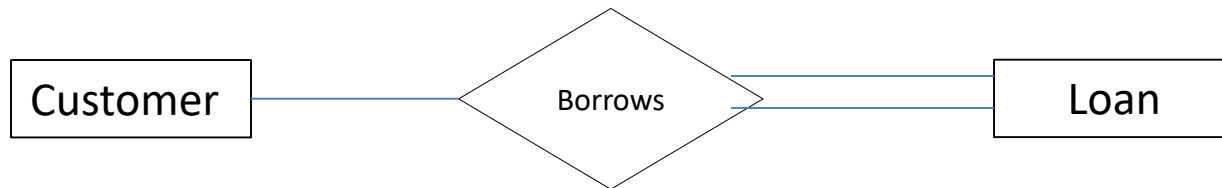
- Many- to- Many: an entity in A is associated with any number of entities in B, and an entity in B is associated with any number of entities in A.

Eg. Multiple People work on Multiple Projects



Participation Constraint

- It defines the number of times in an object class can participate in a relationship. It specifies whether the existence of an entity depends on being related to an entity via relationship.
- a) Total Participation: Each entity must participate in a relationship , it can not exist without.
- b) Partial Participation: Each entity can participate relationship partially.



Keys

- Keys are one of the basic requirements of a relational database model. It is widely used to identify the tuples(rows) uniquely in the table.
 - A key allows us to identify a set of attributes that distinguish relationship from each other.
- a) **Super Key:** The set of attributes that can uniquely identify a tuple is known as Super Key. A super key is a group of single or multiple keys that identifies rows in a table. It supports NULL values.

Eg. Super key= { student_id, First name+last name }

b) **Candidate Key:** The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. It is a subset of Super Key. Candidate Key is the single field or least combination of fields that uniquely identifies each record in the table.

Eg. Candidate key= { {student_id}, {First name+last name} }

Rules

- *It must contain unique values.*
- *It must not contain unique null values.*
- *It contains the minimum number of fields to ensure uniqueness.*

c) Primary Key: There can be more than one candidate key in relation out of which one can be chosen as the primary key.

Eg. Primary key= student_id or First name+last name }

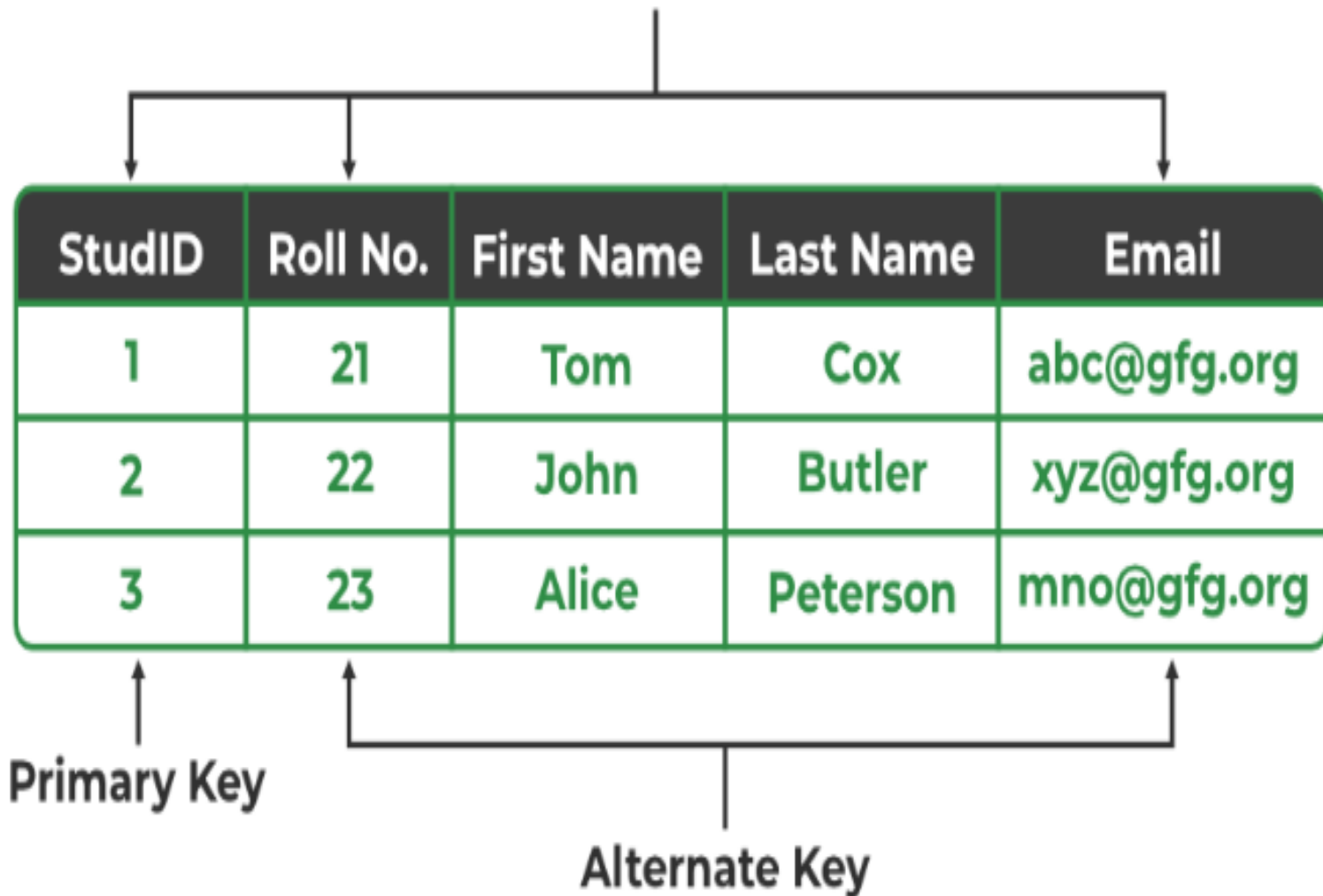
Rules

- *It is a unique key.*
- *It can identify only one tuple (a record) at a time.*
- *It has no duplicate values, it has unique values.*
- *It cannot be NULL.*
- *Primary keys are not necessarily to be a single column; more than one column can also be a primary key for a table.*

d) Alternate Key: The candidate key other than the primary key is called an alternate key.

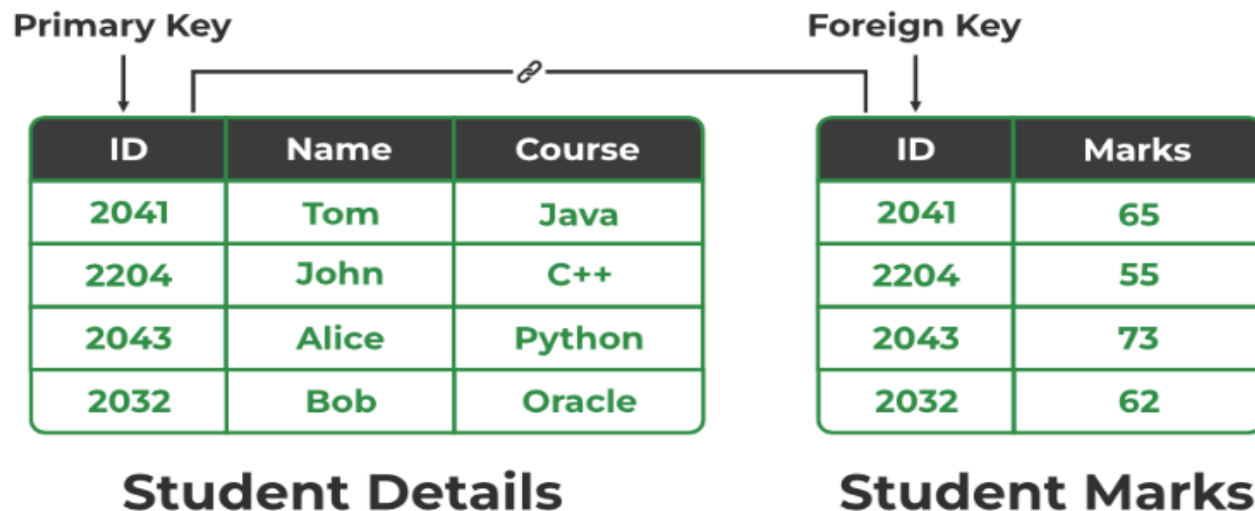
- It is a secondary key.
- It contains two or more fields to identify two or more records. These values are repeated.
- Eg. Alternate Key= Name, and Address is Alternate keys

Candidate Key



e) **Foreign Key**: If an attribute can only take the values which are present as values of some other attribute, it will be a foreign key to the attribute to which it refers. The relation which is being referenced is called **referenced relation** and the corresponding attribute is called **referenced attribute**, the relation which refers to the referenced relation is called **referencing relation** and the corresponding attribute is called **referencing attribute**. The referenced attribute of the referenced relation should be the primary key to it.

- It is a key it acts as a primary key in one table and it acts as secondary key in another table.
- It combines two or more relations (tables) at a time.
- They act as a cross-reference between the tables.

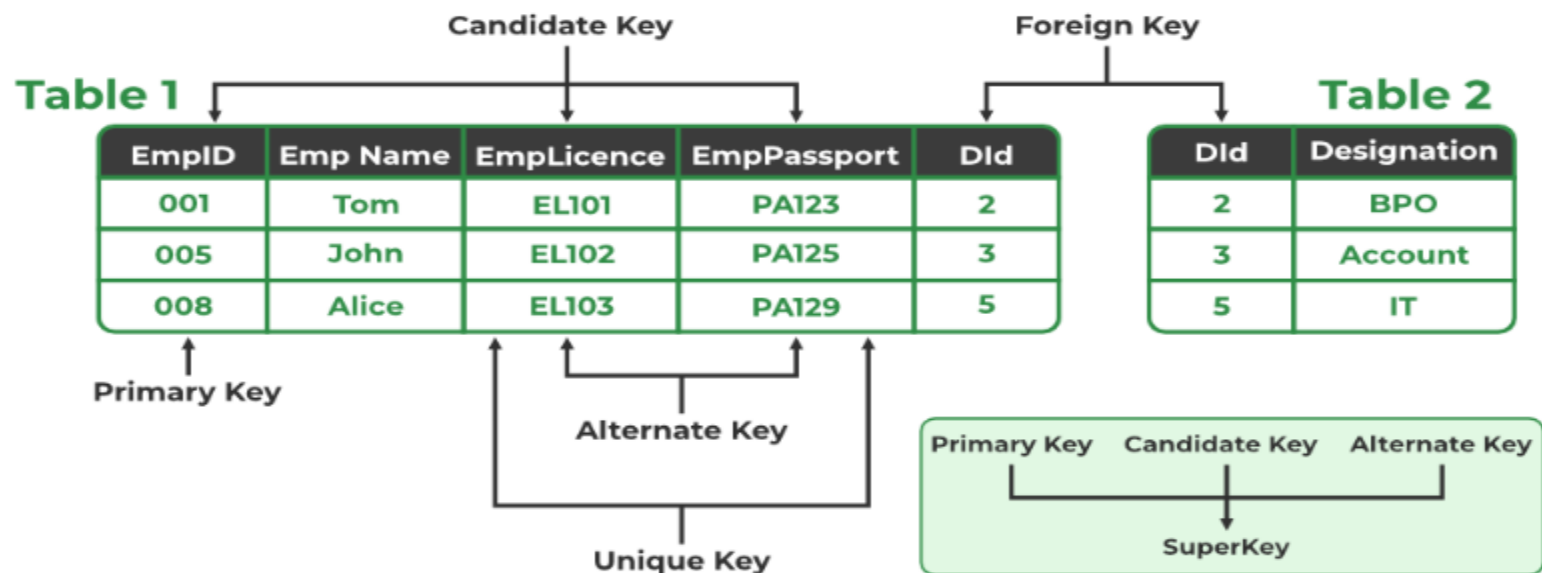


f) **Simple Key**: consist of a single field to uniquely identify a record.

g) **Composite Key**: that consist of two or more attribute that uniquely identifies an entity.

h) **Unique Key**: set of one or more fields of an table that uniquely identify a record in DB.

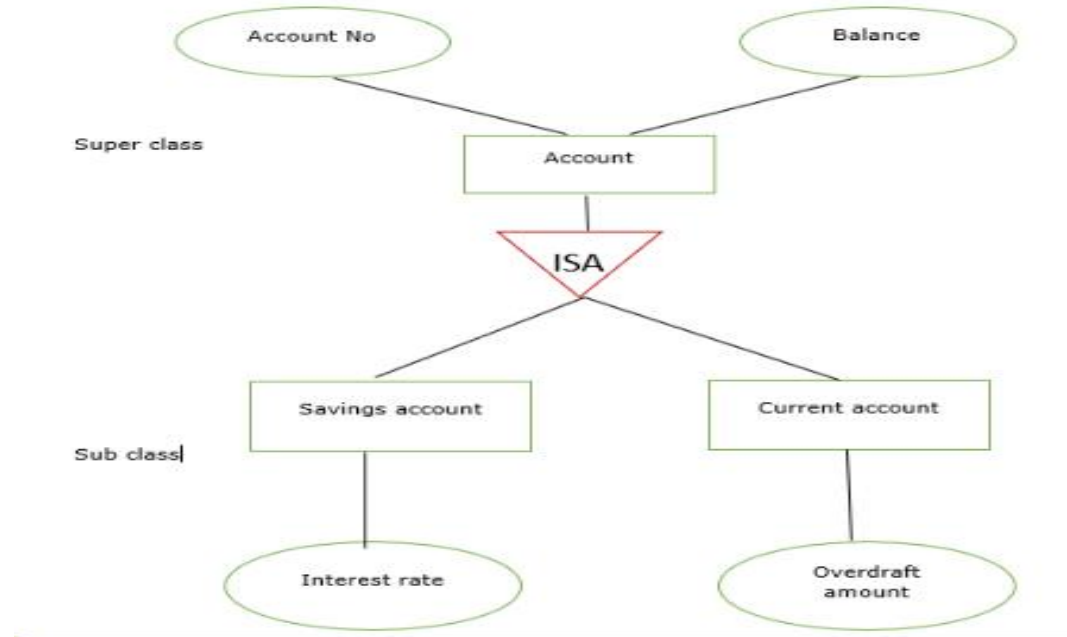
It is like primary key but it can accept only one null value and it cannot have duplicate values.



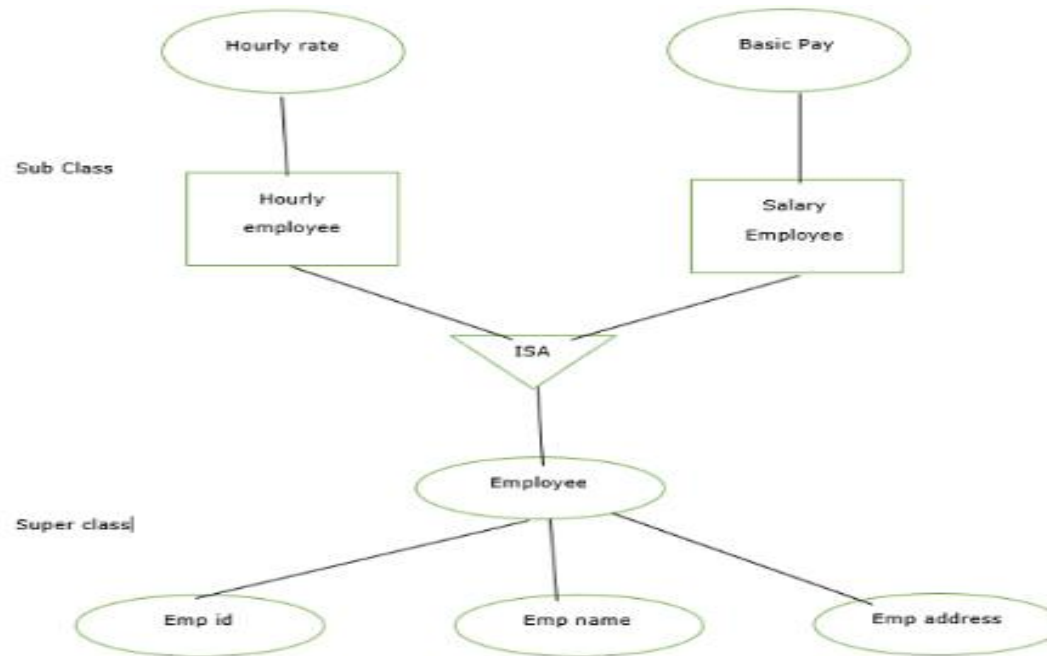
Extended Features of ER Model

a) **Specialization**: The process of designing sub groupings within an entity set is called specialization. It is a top-down process. If an entity set is given with all the attributes in which the instances of the entity set are differentiated according to the given attribute value, then that sub-classes or the sub-entity sets can be formed from the given attribute.

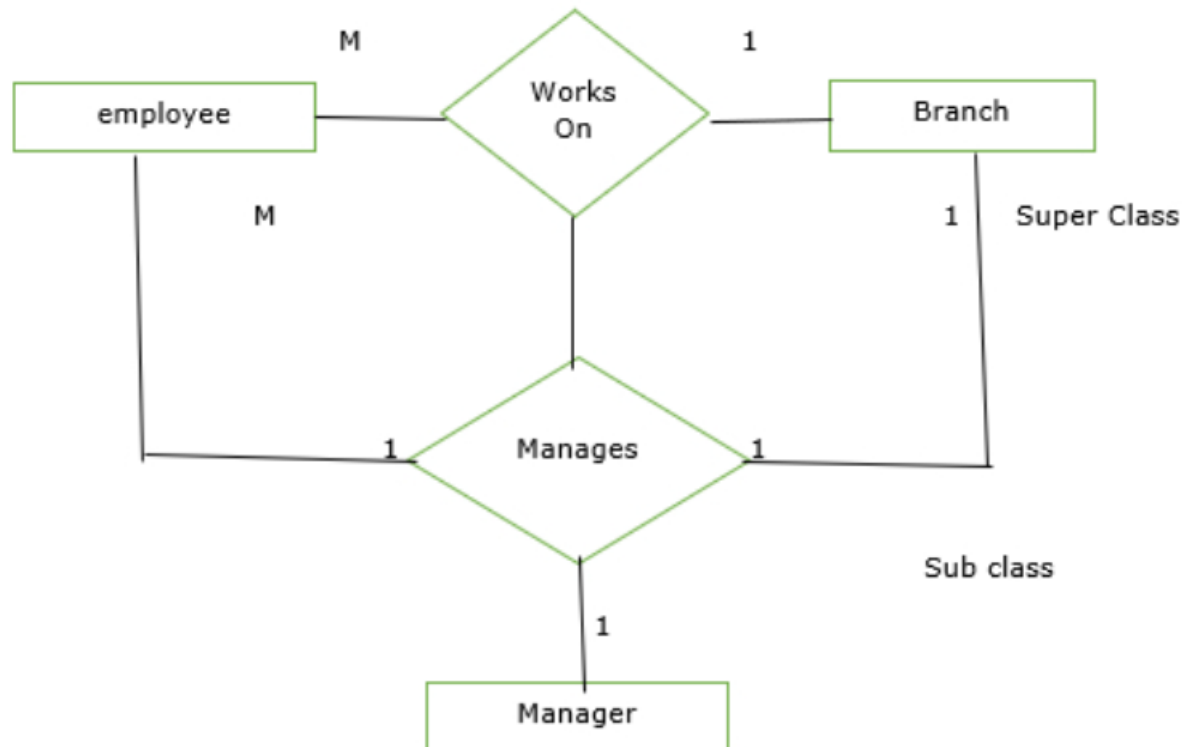
Example: Specialization of a person allows us to distinguish a person according to whether they are employees or customers. Specialization of account creates two entity sets: savings account and current account. In the E-R diagram specialization is represented by triangle components labeled ISA. The ISA relationship is referred as superclass- subclass relationship



- **b) Generalisation:** It is the reverse process of specialization. It is a bottom-up approach.
- It converts subclasses to superclasses. This process combines a number of entity sets that share the same features into higher-level entity sets.
- If the sub-class information is given for the given entity set then, ISA relationship type will be used to represent the connectivity between the subclass and superclass



- **c) Aggregation:** It is an abstraction in which relationship sets are treated as higher level entity sets and can participate in relationships. Aggregation allows us to indicate that a relationship set participates in another relationship set.
- Aggregation is used to simplify the details of a given database where ternary relationships will be changed into binary relationships. Ternary relation is only one type of relationship which is working between three entities.



Design of an ER Database Schema

- Identify entity set
- Identify relationship set
- Identify an associate attributes with entity and relationship type
- Determine attribute domain
- Determine primary key attribute for entity type
- Associate cardinality ratio with relationship type
- Design generalization or specialization hierarchies including constrains
- Design ER diagram

Reduction of ER Model to Tables

- Primary key allow entity sets or relationship set to be expressed uniformly stable which represents the content of the database
- ER diagram can be represented as a collection of table.
- For each entity set and relationship set, there is a unique table which is assigned the name of corresponding entity set or relationship set
- Each table has a number of columns which has unique name
- Converting a ER diagram to a table format is the basis for deriving a relational database design from ER diagram.

Step 1: Entity type becomes a table.

Step 2: All single-valued attribute becomes a column for the table.

Step 3: A key attribute of the entity type represented by the primary key.

Step 4: The multivalued attribute is represented by a separate table.

Step 5: Composite attribute represented by components.

Step 6: Derived attributes are not considered in the table.

